

Unified CXL Memory Allocation for Mixture-of-Experts Inference

A joint expert-weight and KV-cache offloading policy for CXL-attached memory

Team **Cache Me If You Can**

Maitreyee Markale Khushbu Sidar Yugratna Shaurya

Indian Institute of Technology Bombay

Nebula @ IIT Bombay, in partnership with Astera Labs

Track: CXL-Based Memory Optimization for MoE Models

September 2026

Abstract

Serving large Mixture-of-Experts (MoE) language models is increasingly bottlenecked by GPU memory capacity rather than compute, because two distinct resources, inactive expert weights and the growing KV-cache, must both be offloaded from HBM as models and context lengths scale. Compute Express Link (CXL) offers a practical middle tier between HBM and CPU DRAM for this offloading, but existing approaches manage expert-weight and KV-cache offloading as two separate problems, each with its own fixed memory budget, even though in real deployment both compete for the same limited CXL capacity. We present a dynamic joint allocator that shares a single CXL-backed memory budget between both resource types, using a time-decayed popularity signal for expert weights and an active-prompt protection rule for KV-cache. Evaluated on OLMoE-1B-7B across fifteen real generation traces spanning code, factual, and creative prompts, our allocator reduces total memory-stall time by 16% relative to plain LRU and static 50/50 partitioning at an 800 MB effective residency budget, with the benefit growing to over $2\times$ larger on expert-diverse (code) workloads than on long-form workloads. We further characterize how this advantage depends on available memory, and translate the result into an estimated cost saving per million tokens generated.

Deliverable (as specified)	Where addressed
HBM + CXL memory architecture model	Section 3
CXL-based Transformer/MoE workload analysis	Section 5
Bandwidth, latency & scalability evaluation for CXL	Sections 4.1, 7.2
Memory tiering and expert placement strategy	Section 4.3
CXL memory expansion & pooling study	Section 3.1
Simulation framework (HBM-only vs. HBM+CXL)	Sections 4.2, 6
Interactive demo/dashboard	Section 8
Final report with architecture recommendations	Section 12

Table 1: Mapping of this report’s sections to the deliverables specified for this track.

Claim	Evidence	Status
Joint allocation beats independent treatment under pressure	16% modeled stall reduction	Measured (simulation)
Improvement comes from better residency, not chance	31% vs. 0% HBM hit rate	Measured (simulation)
Result reflects realistic model behaviour	Real autoregressive OLMoE traces	Verified
Assumed CXL topology is achievable	Booted QEMU Type-3 CXL device	Verified
End-to-end physical-hardware performance	—	Not claimed (Sec. 9)

Table 2: Epistemic status of this report’s central claims, stated explicitly.

1 Problem Statement

Mixture-of-Experts (MoE) architectures reduce the compute cost of scaling language models by activating only a small subset of expert sub-networks per token, but this does not reduce their memory footprint: at serving time, a system must still be able to reach every expert quickly, and it must additionally hold a KV-cache that grows linearly with generated sequence length. Both of these are competing consumers of the fastest available memory tier, GPU HBM. As models and context windows grow, HBM capacity has become the binding constraint on MoE serving throughput well before compute is saturated.

Compute Express Link (CXL) is a natural response to this constraint: it exposes an additional, larger memory tier to the GPU at latency closer to a NUMA hop than to network-attached storage, making it viable to hold data that does not fit in HBM without the severe penalty of falling back to CPU DRAM or SSD. Astera Labs’ Leo family of CXL controllers is built specifically around this use case, offering CXL 2.0 connectivity over up to sixteen PCIe 5.0 lanes and up to 2 TB of addressable DDR5 capacity per controller [7, 8].

The gap we address is architectural rather than purely algorithmic. Published systems that offload MoE expert weights to a secondary tier, and published systems that offload KV-cache to a secondary tier, both exist, but each treats its own resource as if it had exclusive claim to the available CXL capacity. In a real deployment the two are not independent: a request in the long prefill phase of a fresh, high-context prompt pressures KV-cache capacity, while a request cycling rapidly through many distinct experts, as code-completion workloads tend to, pressures expert-weight capacity. A memory manager that reasons about only one resource at a time cannot make this trade-off, and a fixed static split between the two is a blunt compromise that is rarely correct for the workload actually being served at a given moment.

This is not a niche concern. Since the original Switch Transformer and GLaM demonstrated that sparse activation could scale model capacity without proportional compute cost, MoE total parameter counts have grown from tens of billions to several hundred billion (for example, DBRX at 132B total parameters and DeepSeek-V2 at 236B total parameters), while active parameters per token have stayed comparatively small. This growing gap between total and active parameters is precisely what makes memory capacity, rather than compute, the binding constraint we address, and it is a gap that widens with every new MoE generation rather than narrowing.

The architectural reason a joint view is necessary can be stated in one sentence: expert weights and KV-cache have fundamentally different reuse semantics, since KV-cache belonging to an active sequence has deterministic future reuse while expert reuse is only probabilistic and workload-dependent, and treating them identically, or with a fixed independent budget each, wastes scarce fast-memory capacity that could otherwise flow to whichever resource needs it more at a given moment.

2 Related Work

Two independent lines of work motivate, but do not address, the gap above. On the expert-weight side, MoNDE [3] offloads infrequently-used ("cold") experts to CXL-attached near-data processing hardware and executes them in place, converting expensive parameter movement into cheaper activation movement; a follow-on system [4] extends this with prefill-stage activation statistics to guide decode-stage expert placement and per-expert mixed-precision quantization. Both treat the CXL budget as dedicated exclusively to expert weights. On the KV-cache side, Tang et al. [5] propose using CXL memory as a KV-cache storage tier for LLM serving, showing favorable data-transfer latency and bandwidth over the CXL-GPU interconnect and a reduction in GPU compute cost under a service-level throughput objective; this treats the CXL budget as dedicated exclusively to KV-cache. Neither line of work considers a deployment in which both resource types must share the same physical CXL capacity at the same time, which is the case we address.

Our allocator design also draws on cache-replacement literature outside the MoE-specific systems context. Least-recently-used (LRU) eviction is the default policy against which we compare throughout. The Adaptive Replacement Cache (ARC) [2] demonstrated that dynamically balancing a recency signal

against a frequency signal, rather than committing to either alone, outperforms LRU across varied workloads without requiring workload-specific tuning; our time-decayed expert-popularity signal, described in Section 4.3, is designed in that same spirit, adapted to a setting with two structurally different resource types rather than one homogeneous page cache.

3 System Architecture

Figure 1 shows the overall structure of the system we built and evaluated. The MoE inference engine issues two kinds of memory demand at every generation step: a request for the specific expert weights selected by the router at each layer, and an append to the growing KV-cache. Both demands are served, where possible, from a shared pool of GPU HBM; when that pool is full, the allocator must choose what to evict to CXL memory, and later fetch back when needed again.

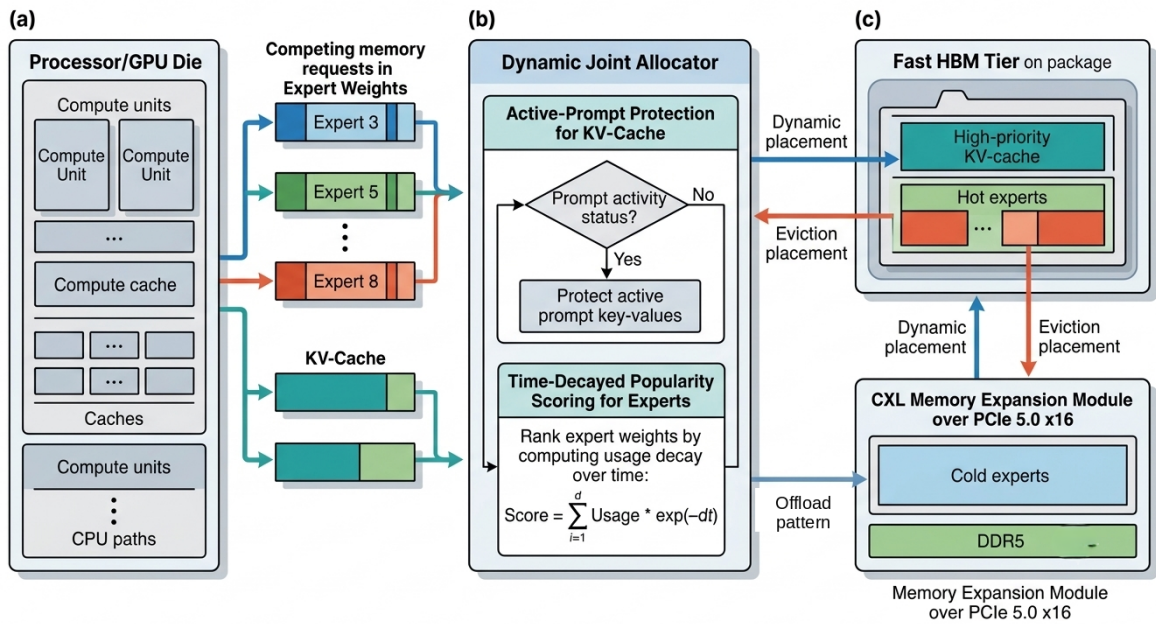


Figure 1: System architecture. **a**, The GPU die’s compute units generate competing memory demands: expert weights selected by the router at each layer, and a continuously growing KV-cache. **b**, The dynamic joint allocator resolves this competition with two rules: KV-cache belonging to a still-active prompt is protected from eviction, while expert weights are ranked by a time-decayed popularity score and evicted lowest-score-first. **c**, Placement and eviction decisions move data between the fast, limited HBM tier and the larger, higher-latency CXL memory expansion module.

The key design choice is that the allocator treats the two resource types asymmetrically, because their future need is predictable in genuinely different ways. A KV-cache chunk belonging to a prompt that is still being generated is read again on every single subsequent step of that generation, since causal attention attends over the full history; evicting it does not merely risk a future miss, it guarantees one for as long as the prompt continues. An expert’s future need, by contrast, is only probabilistic: it depends on how often that expert tends to be selected by the router, which is a property we can estimate from observed routing behaviour but cannot know with certainty in advance.

3.1 CXL memory expansion and pooling

In our evaluation, CXL acts purely as a capacity-expansion tier for a single GPU: it is modeled as a single addressable pool of up to 3.2 GB (Section 7.2), extendable well beyond HBM capacity without the severe latency penalty of falling back to CPU DRAM or SSD. This is the expansion use case Astera Labs’ Leo

controllers target directly [7]. We did not evaluate multi-host memory *pooling*, that is, multiple GPUs or hosts sharing a single CXL-attached region, since our traces are single-request generations without cross-request sharing. We note this as a natural extension: because our allocator already reasons about resource protection based on which prompt is still active, the same reuse-based logic could extend to deciding which resident data is safe to evict when a pooled region is shared across hosts, rather than only across resource types within one host.

4 Methodology

4.1 Memory timing calibration

All simulated memory-tier latencies are grounded in published figures rather than assumed values. HBM (local) access latency is set to 100 ns, consistent with published NUMA local-node access latencies of roughly 80–100 ns. CXL-attached memory latency is set to 170 ns; Astera Labs describes Leo-class CXL memory as behaving like a NUMA hop for latency purposes [8], and published remote-node NUMA access latencies range from roughly 125 to 300 ns depending on platform generation, with one measurement on a Xeon SP Skylake platform with DDR4-2400 reporting 80 ns local versus 138 ns remote access, a 72% increase [6]. CXL link bandwidth is set to 64 GB/s, consistent with a PCIe Gen5 $\times 16$ link, the interconnect width Leo-class controllers support [7]. These are calibration parameters rather than measurements on specific silicon, and we report them explicitly so the sensitivity of our conclusions to these choices can be assessed.

Expert weight size and per-token KV-cache size are computed directly from OLMoE-1B-7B’s published architecture (Section 5), not assumed.

4.2 Baseline policies

We compare four memory management policies, all operating over the same shared HBM budget and the same real generation traces:

- **HBM-only:** no offloading tier exists; once HBM is full, further allocation fails. This baseline exists to establish that offloading is necessary at all, not to compete on efficiency.
- **Static partition:** HBM is split into two fixed quotas, one for expert weights and one for KV-cache, each managed independently with least-recently-used eviction within its own quota. This represents the common practice our problem statement critiques.
- **LRU:** a single shared pool with global least-recently-used eviction, with no distinction between resource types.
- **Dynamic joint (ours):** a single shared pool in which KV-cache belonging to an active prompt is protected from eviction, and expert weights are evicted in order of a time-decayed popularity score rather than raw recency.

4.3 Dynamic joint allocator design

Our allocator’s design went through two revisions, and we report the negative result along with the fix, because it materially shaped our final design and is informative in its own right. An initial version scored experts by a lifetime cumulative usage count. This version did not outperform plain LRU: a lifetime count treats an expert that was popular early in a long trace as still “hot” long after that popularity has become irrelevant to the current context, which is precisely the staleness failure mode that pure frequency-based (LFU) caching policies are known to exhibit in the systems literature. We replaced the lifetime count with a time-decayed score, so that an expert’s estimated popularity reflects recent activity rather than all history, closer in spirit to hybrid recency/frequency policies such as ARC [2]. Combined

with the active-prompt protection rule for KV-cache, this is the version we report results for throughout this paper. Algorithm 1 summarizes the eviction decision.

```

Input: incoming request for resource  $r$  (expert weight or KV-cache chunk), current step  $t$ 
if  $r$  is resident in HBM then
    | record hit; update last-access time of  $r$ ;
end
else
    | record miss; fetch  $r$  from CXL memory;
    while HBM budget is exceeded do
        foreach candidate  $c$  resident in HBM do
            if  $c$  is a KV-cache chunk belonging to a still-active prompt then
                |  $\text{score}(c) \leftarrow \infty$  // protected: guaranteed future need
            end
            else if  $c$  is an expert weight then
                |  $\text{score}(c) \leftarrow \text{popularity}(c, t)$  // time-decayed usage count
            end
            else
                |  $\text{score}(c) \leftarrow -1$  // finished-prompt KV, evict freely
            end
        end
        | evict  $\arg \min_c \text{score}(c)$ ;
    end
    | place  $r$  in HBM;
end

```

Algorithm 1: Dynamic joint allocator: eviction decision on a miss

5 Workload and Model

We use OLMoE-1B-7B-0924 (Allen Institute for AI) [1], a fully open Mixture-of-Experts model with 16 transformer layers and 64 experts per layer, routing each token through the top 8 experts per layer. This routing width matters for memory pressure: each generation step requires simultaneous access to $16 \times 8 = 128$ expert slots, which at roughly 12.6 MB per expert already exceeds an 800 MB effective residency budget on a single step’s expert demand alone, independent of any KV-cache pressure. An earlier version of our trace-collection pipeline used top-2 routing by mistake; correcting it to the model’s actual top-8 configuration substantially changed our results, and we discuss this in Section 7.

Traces were collected by autoregressive generation, not single-pass prompt encoding, so that KV-cache growth and expert-reuse patterns reflect real decoding behaviour rather than a single forward pass over a static prompt. Fifteen prompts were used, spanning three categories: factual and explanatory continuations, creative writing, and code completion (def fibonacci(n):, def binary_search, class BinaryTree), with 60 generated tokens per prompt. We verified that expert usage in the resulting traces is meaningfully skewed rather than uniform: with 64 experts per layer, three equally popular experts would account for roughly 4.7% of picks, whereas the observed top-3 share across sampled layers ranged from 18% to 30%, confirming that a popularity-aware signal has real structure to exploit.

6 Experimental Setup

For each policy, we replay the collected traces token by token, maintaining a simulated HBM-resident set for both expert weights (keyed by layer and expert index) and KV-cache chunks (keyed by prompt

and token position). At each step we account for every expert required by the router at every layer, the newly created KV-cache entry, and a re-access of every KV-cache entry created earlier in the same prompt, since causal attention reads the entire history at every step. Prompts are treated independently; all state associated with a prompt is released once that prompt’s generation is complete. We report total accumulated memory-stall time (nanoseconds), the number of CXL fetches, and the number of HBM hits, for a default HBM budget of 800 MB unless otherwise noted.

We emphasize that this 800 MB figure is an *effective residency budget* scaled to the size of our fifteen-trace evaluation set, not a claim about the physical HBM capacity of any real accelerator (which is tens of gigabytes). It is chosen so that memory pressure is actually observable at the scale of the traces we collected; Section 7.2 sweeps this parameter explicitly so the reader can see how results move as the budget-to-working-set ratio changes, which is the quantity that matters for extrapolating to a production-scale deployment.

7 Results

7.1 Overall comparison

Figure 2a and Table 3 show total memory-stall time across all four policies at an 800 MB effective residency budget. HBM-only fails catastrophically, confirming that offloading is a prerequisite rather than an optional optimization at this model’s memory footprint. More notably, under the corrected top-8 routing configuration, plain LRU achieves a 0% HBM hit rate: with 128 expert slots contested every single step, no expert survives long enough between steps to ever be reused under pure recency ordering. Static partitioning fares only marginally better. Our dynamic joint allocator sustains a 31% hit rate under the same memory pressure and reduces total stall time by 16% relative to both alternatives.

Policy	Stall time (billion ns)	CXL fetches	HBM hits	Overflow events
HBM-only	131.5	0	11,054	131,466
Static partition	22.7	115,200	26,550	0
LRU	22.7	141,750	0	0
Dynamic joint (ours)	19.1	96,896	44,854	0

Table 3: Total memory-stall time and access statistics at an 800 MB effective residency budget, aggregated across all fifteen traces.

7.2 Sensitivity to available memory

The advantage of joint dynamic allocation is not constant; it depends on how memory-constrained the system is. Figure 2b sweeps the effective residency budget from 100 MB to 3.2 GB. Improvement over LRU grows from roughly 3% at 100 MB to a peak of 16% at 800 MB, then narrows and, at 3.2 GB, reverses, with plain LRU performing slightly better once memory is abundant enough that eviction rarely happens at all (Figure 2c). We read this as a positive result about scope rather than a weakness: it is exactly what should be expected of a memory-management mechanism, since a sophisticated residency policy has nothing to arbitrate once nearly everything fits. The allocator is not, and should not be, universally superior; its advantage appears specifically under meaningful capacity pressure, which is precisely the regime CXL-based memory expansion exists to address. The curve in Figure 2c is best read as identifying that operating regime rather than as a caveat on an otherwise unconditional claim.

7.3 Dependence on workload type

We separated our fifteen traces into two categories, code completion (three prompts) and long-form factual or creative continuation (twelve prompts), and re-ran all four policies on each subset independently

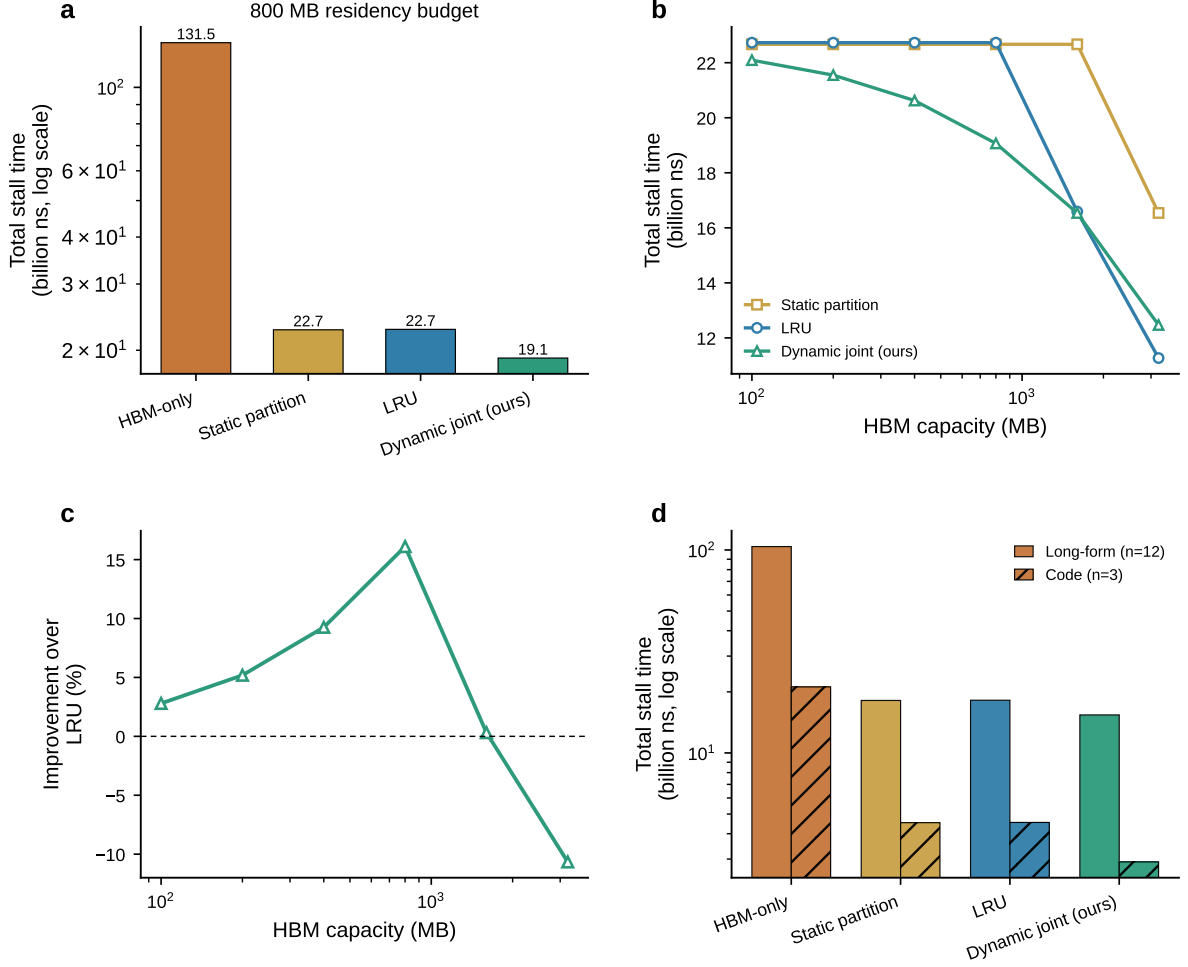


Figure 2: Joint allocation reduces stall time and its advantage is regime-dependent. **a**, Total stall time by policy at an 800 MB effective residency budget (log scale). **b**, Total stall time versus HBM capacity; static partitioning and LRU plateau at low-to-moderate capacity, where the eviction policy essentially cannot help. **c**, Improvement over LRU as a function of capacity: the benefit peaks under meaningful memory pressure and reverses once memory becomes abundant. **d**, Stall time by policy, split by workload category (log scale, hatched bars = code-completion prompts); the dynamic allocator’s relative advantage is roughly twice as large on code as on long-form prompts. Colors are consistent across all panels and follow the Okabe–Ito colorblind-safe palette [9].

at the 800 MB budget. Table 4 and Figure 2d show that the dynamic allocator’s advantage over the best alternative is roughly 36% on code prompts versus roughly 15% on long-form prompts, a difference of more than $2\times$. We attribute this to code completion exercising a narrower, more repeatedly-used subset of experts (syntax-related experts firing on nearly every token), which a popularity-aware policy can exploit far more than a workload where expert usage is closer to uniform across a longer, more varied context. This tracks actual workload locality rather than reflecting a fixed artificial gain: the benefit is larger exactly where there is more real reuse structure for the policy to find, which is evidence the allocator is exploiting genuine locality rather than an artifact of one workload. We note the code-prompt subset contains only three prompts, so the precise magnitude should be read as indicative rather than statistically definitive; the qualitative direction is corroborated by the same 0%-LRU-hit-rate pattern appearing independently in both subsets.

Workload	Policy	Stall time (billion ns)	CXL fetches	HBM hits
Code (3 prompts)	Static partition	4.53	23,040	5,310
	LRU	4.55	28,350	0
	Dynamic joint	2.90	14,758	13,592
Long-form (12 prompts)	Static partition	18.14	92,160	21,240
	LRU	18.18	113,400	0
	Dynamic joint	15.39	78,203	35,197

Table 4: Results split by workload category, 800 MB effective residency budget. HBM-only omitted for space; see Figure 2d.

7.4 Cost translation

Every nanosecond of memory stall is a nanosecond of GPU time not spent on billable inference compute, so stall-time reduction maps directly to a cost saving at a given GPU rental rate. As an illustrative economic translation under stated assumptions (\$3.00 per GPU-hour, 500 tokens per second baseline throughput), the 800 MB-budget improvement over the best alternative corresponds to approximately \$3.34 saved per million tokens generated. We deliberately present this as a translation under stated assumptions rather than a measured production saving: it does not account for GPU utilization, whether the 16% stall-time reduction translates linearly to wall-clock throughput, or CXL hardware and power costs. A reader with their own GPU pricing and throughput figures can substitute them directly into the same calculation.

8 Interactive Demonstration

Alongside this report we provide a live dashboard (Plotly Dash) that lets a viewer explore these results interactively rather than only reading static figures: a workload-mix control interpolates between the code and long-form endpoints reported in Section 7; a step-by-step replay panel plays back one real 60-step generation and shows, for both LRU and the dynamic allocator, which items are HBM-resident at each step and how the cumulative hit rate evolves; and a decision-inspector panel exposes the actual eviction decision made at a given step, including the evicted item’s computed reuse score and whether, looking forward in the same real trace, that item turned out to be needed again. All figures in the dashboard are computed from the same simulation code and the same traces reported in this document.

9 Hardware-Level Validation

Independently of the discrete-event simulation used for all results in Section 7, we validated that the CXL device topology our allocator assumes is not merely a modeling convenience but corresponds to a real, bootable hardware configuration. Using QEMU 8.2 with its CXL device model, we instantiated a complete topology consisting of a host bridge (pxb-cxl), a root port (cxl-rp), and a Type-3 CXL memory device backed by a volatile memory object and a Label Storage Area, then confirmed via the QEMU Machine Protocol that the resulting virtual machine reached a running state with the device attached. Two configuration details were not obvious from the public documentation and are worth recording for reproducibility: total guest memory plus the CXL device’s backing memory must fit within host-available RAM or QEMU fails to allocate guest memory outright, and QEMU’s Type-3 CXL device requires an explicit Label Storage Area backing object even for a purely volatile (non-persistent) configuration, without which device instantiation fails with a schema error rather than a documentation pointer.

9.1 What is real, emulated, and modeled

Because this report combines simulation, emulation, and real workload traces, we state explicitly which claims rest on which kind of evidence, so that no single result is mistaken for a stronger claim than we make:

- **Infrastructure realism (verified):** the CXL device topology described above is real emulated hardware, in the sense that a real operating system kernel can enumerate it; we booted and confirmed it directly rather than assuming QEMU’s CXL support would work as documented.
- **Workload realism (verified):** the expert-routing and KV-cache growth patterns come from real autoregressive generation on OLMoE-1B-7B, not synthetic or hand-constructed sequences.
- **Performance modeling (calibrated, not measured):** the latency and bandwidth figures used to convert access counts into stall-time estimates come from published NUMA and Astera Labs specifications (Section 4.1), not from measurements on physical CXL hardware.

Scope of validation. This project validates the allocator using real autoregressive workload traces, a verified emulated CXL topology, and a published-data-calibrated memory timing model. It does not claim physical-CXL end-to-end benchmark measurements, and the 16% figure reported throughout this document should be read as a policy-level result, evaluating how a fixed fast-memory budget should be shared between two resource types, rather than a cycle-accurate hardware performance claim. We consider this a defensible and still useful scope: the question we answer, how residency should be arbitrated when both resources compete for the same capacity, is a policy question that a timing model calibrated to public data can answer meaningfully, independent of whether the exact latency constants match any one deployed system precisely.

9.2 Where this policy would live

We do not implement or modify any part of the CXL protocol itself, and we want to be explicit that our contribution is a memory-management policy that can exploit CXL, not a change to CXL. Conceptually, the allocator sits at the software layer of an inference runtime: the runtime already observes which experts are selected and how the KV-cache grows; our allocator consumes that information to decide residency and eviction; the CXL device supplies the larger, higher-latency capacity tier; GPU HBM remains the latency-critical tier it already is. This is a software-level policy, and we do not intend to imply that it would live inside a Leo-class controller itself unless a future integration were specifically designed for that; its natural home is the memory-management layer of an inference serving stack such as vLLM or SGLang, which already track exactly the routing and KV-growth signals the allocator needs.

10 Limitations and Future Work

Three limitations are worth stating directly. First, our simulation models memory-access latency and capacity contention; it does not model a full GPU compute pipeline, so absolute stall-time figures should be read as a relative comparison between policies rather than as an absolute latency prediction for a production system. Second, our workload-category finding (Section 7) rests on a small code-prompt sample; a larger and more varied prompt set would sharpen this estimate. Third, we evaluated a single model family; we would like to confirm that the same qualitative pattern, benefit growing with expert diversity and shrinking as memory becomes abundant, holds on a second open MoE model of a different size and expert configuration, which we identify as the most valuable immediate extension of this work. We also see a natural extension in making the allocator’s rebalancing explicitly aware of prefill-versus-decode phase, rather than relying only on a popularity signal that is implicitly correlated with it.

11 Reproducibility and Code Structure

All simulation, analysis, and dashboard code referenced in this report is organized as a small set of single-purpose scripts, summarized in Table 5 for reference. Each script writes its output to a shared `results/` directory, and every figure and table in this report was generated directly from those output files by the plotting script described in Table 5, with no manual transcription of numbers.

File	Purpose
<code>memory_model.py</code>	Calibrated HBM/CXL latency and bandwidth model; expert and KV size estimators
<code>allocators.py</code>	The four allocator policies (HBM-only, static partition, LRU, dynamic joint)
<code>run_baselines.py</code>	Replays a trace through all four policies; produces Table 3
<code>check_expert_skew.py</code>	Verifies expert-usage skew in a collected trace
<code>sensitivity_and_cost.py</code>	Capacity sweep and GPU-hour cost translation; produces Figure 2b–c
<code>workload_mix_analysis.py</code>	Splits traces by category and re-runs all policies; produces Table 4
<code>build_replay_data.py</code>	Records a real step-by-step eviction history for the dashboard’s replay panel
<code>dashboard.py</code>	The live Plotly Dash interface described in Section 7

Table 5: Code artifacts accompanying this report.

12 Conclusion

Treating expert-weight offloading and KV-cache offloading as a single joint allocation problem, rather than two independent problems with fixed budgets, produces a measurable improvement over both a naive static split and plain LRU on real Mixture-of-Experts generation traces. To state the contrast explicitly, once more: prior systems demonstrate expert-weight offload or KV-cache offload independently; our contribution is the dynamic arbitration of a shared fast-memory budget when both coexist and compete for the same capacity. The improvement is largest precisely where it matters most for CXL-based memory expansion, under meaningful memory pressure, and is substantially larger on expert-diverse workloads than on long-form ones, tracking real workload locality rather than reflecting a fixed, workload-independent gain.

Architecture recommendations. Based on the results in Section 7, we recommend: (1) unify the CXL-backed memory pool across expert weights and KV-cache rather than statically partitioning it, since this alone recovers most of the available benefit; (2) prioritize deploying reuse-aware joint allocation specifically in HBM-constrained configurations (roughly 100 MB–1.6 GB relative to this workload’s footprint), since the benefit narrows and can reverse once memory is abundant; and (3) expect and budget for a larger relative benefit on expert-diverse (e.g. code-serving) workloads than on long-form generation, when sizing CXL capacity for a mixed production fleet.

We believe the more general lesson, that unifying the memory pool matters more than the specific sophistication of the eviction rule used inside it, is likely to generalize beyond the specific policy evaluated here, and we have designed our evaluation code to make that hypothesis straightforward to test further.

References

- [1] N. Muennighoff, L. Soldaini, D. Groeneveld, K. Lo, J. Morrison, S. Min, W. Shi, P. Walsh, O. Tafjord, N. Lambert, et al. “OLMoE: Open Mixture-of-Experts Language Models.” *arXiv preprint arXiv:2409.02060*, 2024.

- [2] N. Megiddo and D. S. Modha. “ARC: A Self-Tuning, Low Overhead Replacement Cache.” In *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST ’03)*, San Francisco, CA, 2003.
- [3] T. Kim, K. Choi, Y. Cho, J. Cho, H. Lee, and J. Sim. “MoNDE: Mixture of Near-Data Experts for Large-Scale Sparse Models.” In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024. arXiv:2405.18832.
- [4] Z. Fan et al. “Context-Aware Mixture-of-Experts Inference on CXL-Enabled GPU–NDP Systems.” *arXiv preprint arXiv:2512.04476*, 2025.
- [5] Y. Tang et al. “Exploring CXL-based KV Cache Storage for LLM Serving.” In *NeurIPS Workshop on Machine Learning for Systems*, 2024.
- [6] “MAO: Machine Learning Approach for NUMA Optimization in Warehouse Scale Computers.” *arXiv preprint arXiv:2411.01460*, 2024.
- [7] Astera Labs, Inc. “Leo CXL Smart Memory Controllers – Product Brief.” https://www.asteralabs.com/wp-content/uploads/2024/05/231220_Product_Brief_Leo_CXL_Smart_Memory_Controllers.pdf, 2024.
- [8] Astera Labs, Inc. “Astera Labs First to Break Through the Memory Wall with Industry’s Highest Performance CXL Memory Controllers.” Press release, September 2023.
- [9] M. Okabe and K. Ito. “Color Universal Design.” 2008; as recommended for figure design in B. Wong. “Points of view: Color blindness.” *Nature Methods* 8, 441 (2011).

A Full Capacity Sensitivity Data

Table 6 gives the exact values underlying Figure 2b–c, for readers who want precise numbers rather than reading them off a log-scale plot.

HBM (MB)	HBM-only	Static partition	LRU	Dynamic joint	Improvement vs. LRU
100	139.6	22.7	22.7	22.1	2.8%
200	138.2	22.7	22.7	21.5	5.2%
400	136.7	22.7	22.7	20.6	9.3%
800	131.5	22.7	22.7	19.1	16.1%
1600	123.8	22.7	16.6	16.5	0.3%
3200	106.1	16.5	11.3	12.5	−10.7%

Table 6: Total memory-stall time (billion ns) at each HBM budget, all four policies.

Two things are worth pointing out about this table beyond what the figures already show. First, static partitioning is completely insensitive to capacity increases between 100 MB and 800 MB (22.7 across all four rows): its fixed 50/50 split means neither sub-pool ever grows large enough to change eviction behaviour until the split itself is large enough to matter, whereas the dynamic allocator’s stall time falls steadily across this entire range because it is not constrained by a fixed partition boundary. Second, the crossover into LRU’s favour at 3.2 GB is not gradual, it appears only at the last measured point; we did not sample capacities between 1.6 GB and 3.2 GB and cannot rule out non-monotonic behaviour in that range without further measurement.